NPTEL ONLINE CERTIFICATION COURSES

Course Name: Software Project Management

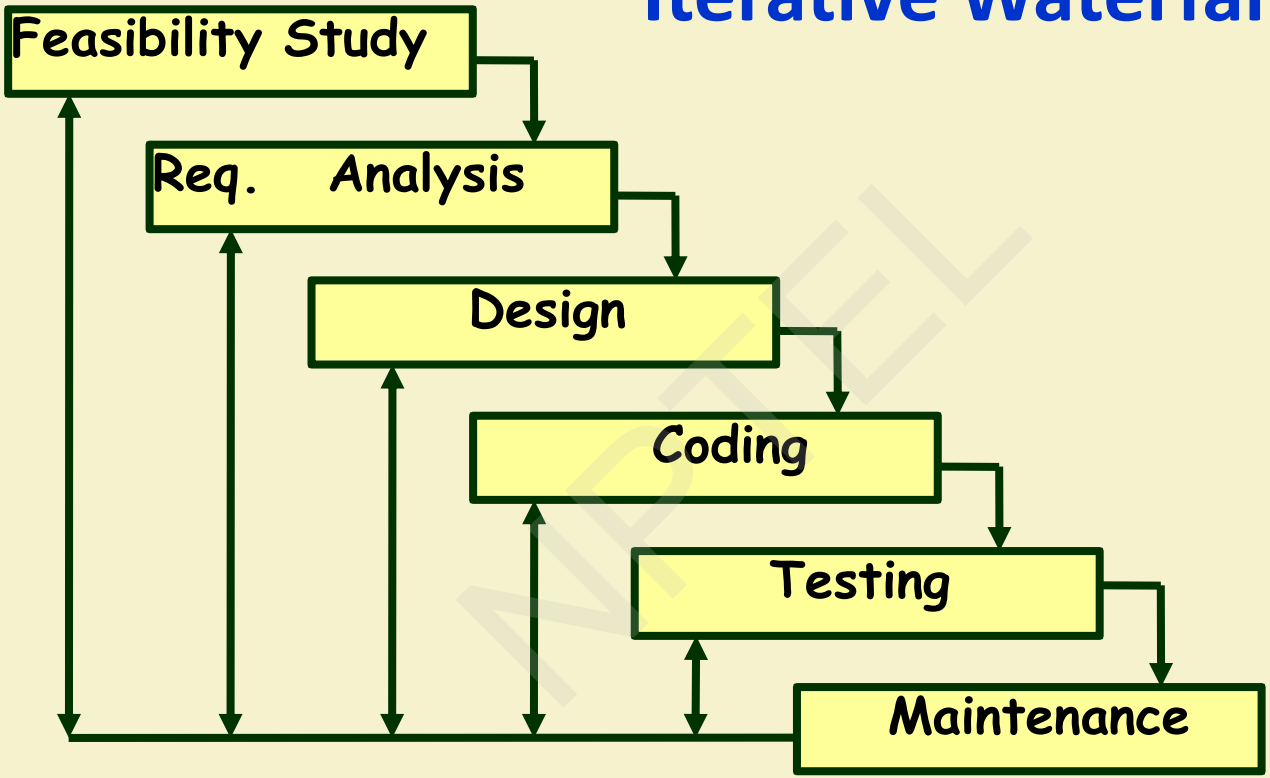
Faculty Name: RAJIB MALL

Department : Computer Science and Engineering

TOPIC: Introduction

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Iterative Waterfall Model (CONT.)



Waterfall Strengths

- Easy to understand, easy to use, especially by inexperienced staff
- Milestones are well understood by the team
- Provides requirements stability during development
- Facilitates strong management control (plan, staff, track)

Waterfall Deficiencies

- All requirements must be known upfront – in **most projects requirement change occurs after project start**
- Can give a false impression of progress
- Integration is one big bang at the end
- Little opportunity for customer to pre-view the system.

When to use the Waterfall Model?

- Requirements are well known and stable
- Technology is understood
- Development team have experience with similar projects

Classical Waterfall Model (CONT.)

- Irrespective of the life cycle model actually followed:
  - The documents should reflect a classical waterfall model of development.
  - **Facilitates comprehension of the documents.**

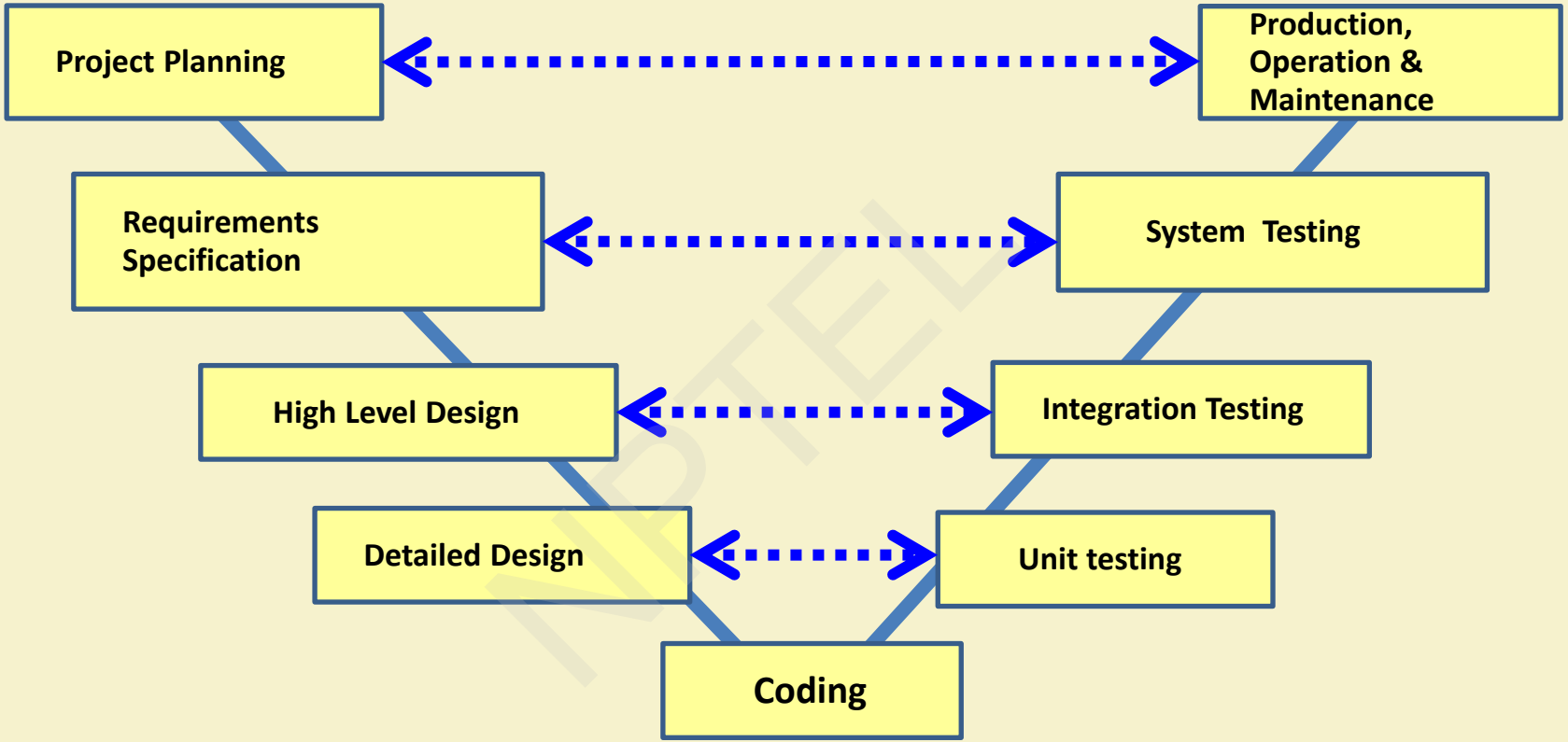
Classical Waterfall Model (CONT.)

- Metaphor of mathematical theorem proving:
  - –A mathematician presents a proof as a single chain of deductions,
    - Even though the proof might have come from a convoluted set of partial attempts, blind alleys and backtracks.

V Model

V Model

- It is a variant of the Waterfall
  - emphasizes verification and validation
  - V&V activities are spread over the entire life cycle.
- In every phase of development:
  - Testing activities are planned in parallel with development.


Project Planning
Production, Operation & Maintenance
Requirements Specification
System Testing
High Level Design
Integration Testing
Detailed Design
Unit testing
Coding

V Model Steps

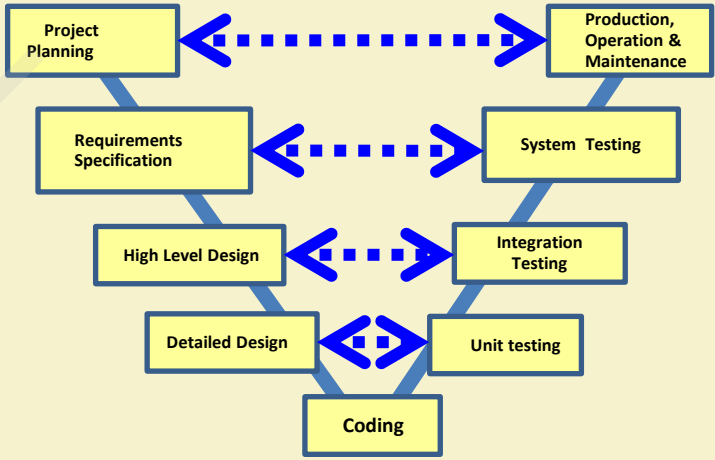
- Planning
- Requirements Analysis and Specification
- High-level Design
- Detailed Design
- System test design
- Integration Test dsign
- Unit test design

V Model: Strengths

- Starting from early stages of software development:
  - **Emphasizes planning for verification and validation of the software**
- Each deliverable is made testable
- Easy to use

V Model Weaknesses

- Does not support overlapping of phases
- Does not handle iterations or phases
- Does not easily accommodate later changes to requirements
- Does not provide support for effective risk handling



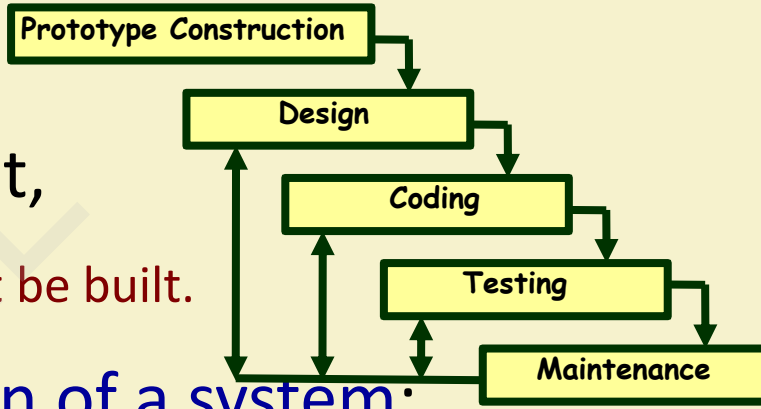
When to use V Model

- Natural choice for systems requiring high reliability:
  - Embedded control applications, safety-critical software
- All requirements are known up-front
- Solution and technology are known

Prototyping Model

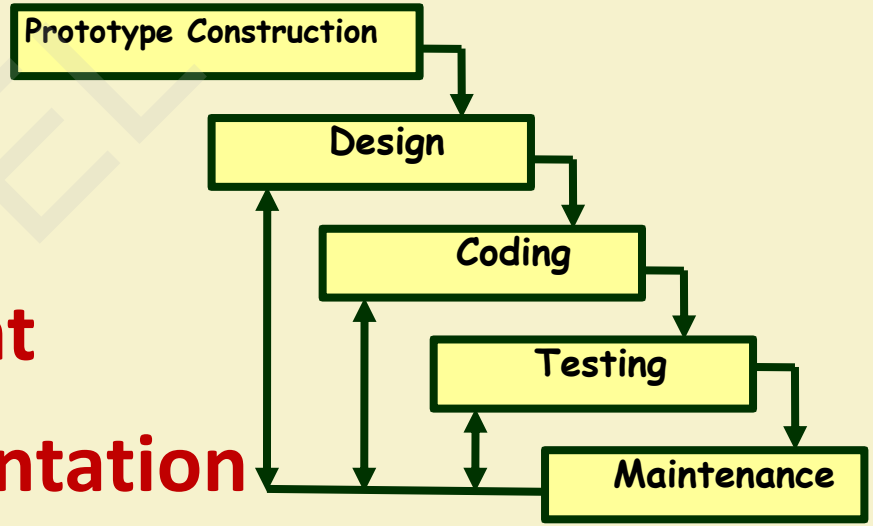
Prototyping Model

- A derivative of waterfall model.
- Before starting actual development,
  - A working prototype of the system should first be built.
- A prototype is a toy implementation of a system:
  - Limited functional capabilities,
  - Low reliability,
  - Inefficient performance.



Reasons for prototyping

- **Learning by doing:** useful where requirements are only partially known
- **Improved communication**
- **Improved user involvement**
- **Reduced need for documentation**
- **Reduced maintenance costs**



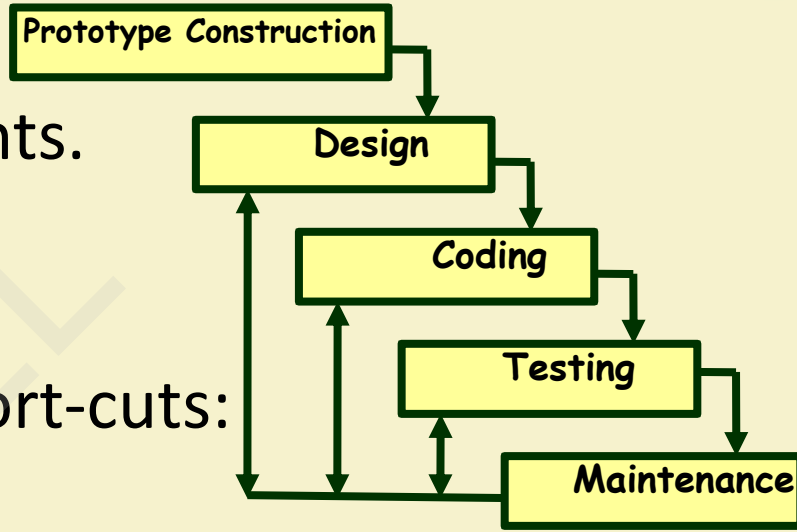
Reasons for Developing a Prototype

- **Illustrate to the customer:**
  - –input data formats, messages, reports, or interactive dialogs.
- **Examine technical issues associated with product development:**
  - –Often major design decisions depend on issues like:
    - Response time of a hardware controller,
    - Efficiency of a sorting algorithm, etc.

Prototyping Model (CONT.)

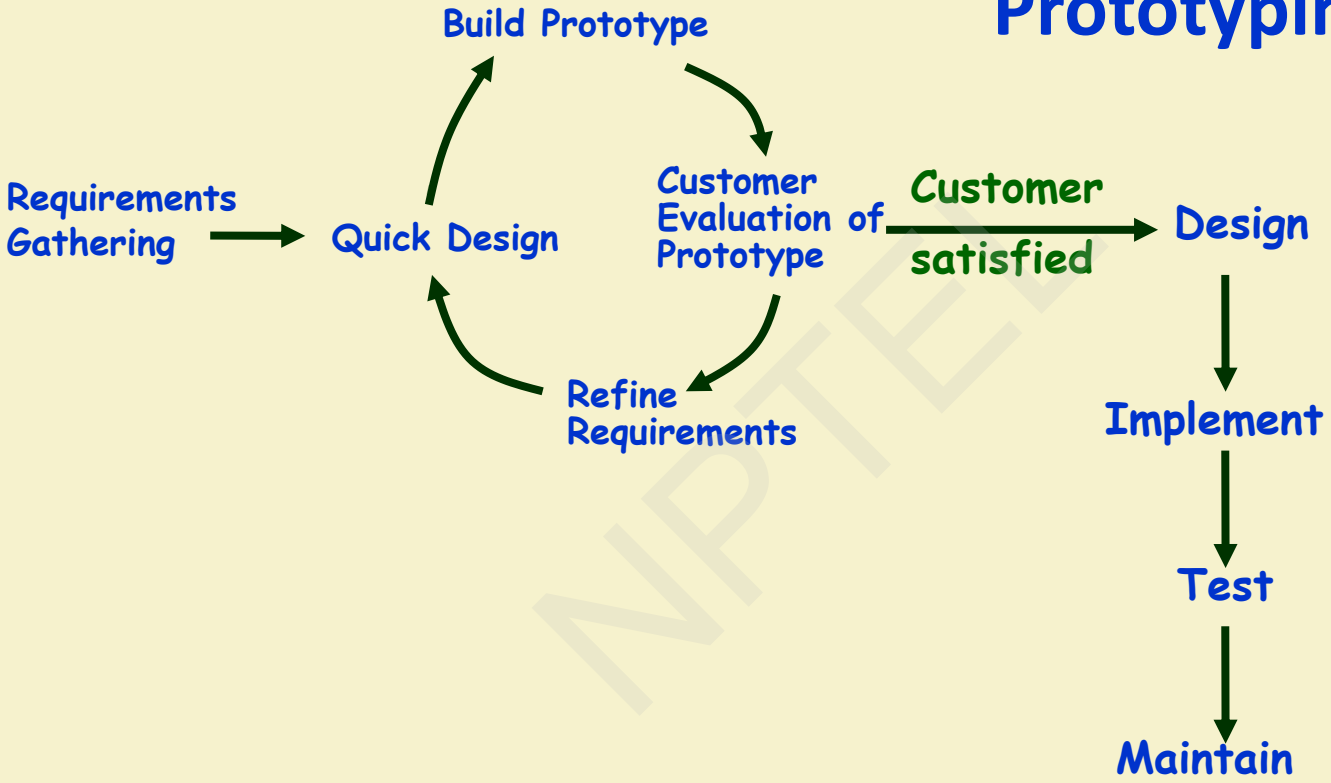
- Another reason for developing a prototype:
  - **It is impossible to "get it right" the first time,**
  - We must plan to throw away the first version:
    - If we want to develop a good software.

Prototyping Model



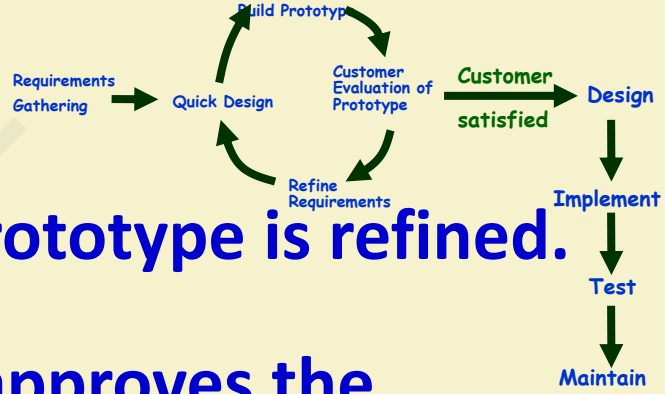
- Start with approximate requirements.
- Carry out a quick design.
- Prototype is built using several short-cuts:
  - –Short-cuts might involve:
    - Using inefficient, inaccurate, or dummy functions.
    - A table look-up rather than performing the actual computations.

Prototyping Model (CONT.)



Prototyping Model (CONT.)

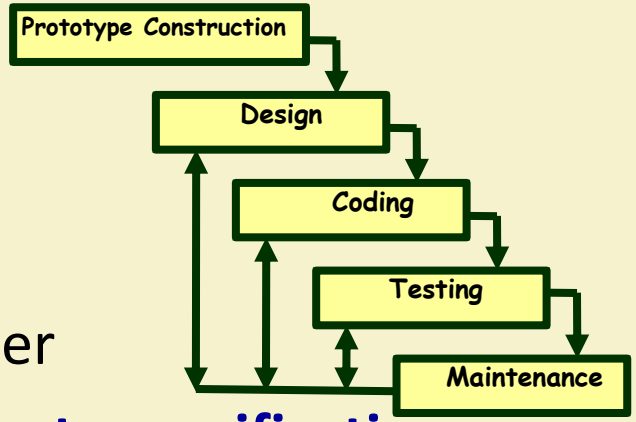
- The developed prototype is submitted to the customer for his evaluation:



  - **Based on the user feedback, the prototype is refined.**
  - **This cycle continues until the user approves the prototype.**

- The actual system is developed using the waterfall model.

Prototyping Model



- Requirements analysis and specification phase becomes redundant:
  - Final working prototype (incorporating all user feedbacks) serves as an **animated requirements specification.**
- **Design and code for the prototype is usually thrown away**:
  - However, experience gathered from developing the prototype helps a great deal while developing the actual software.

Prototyping Model (CONT.)

- Even though construction of a working prototype model involves additional cost --- **overall development cost usually lower for:**
  - Systems with unclear user requirements,
  - Systems with unresolved technical issues.
- Many user requirements get properly defined and technical issues get resolved:
  - These would have appeared later as change requests and resulted in incurring massive redesign costs.

Prototyping: advantages

- The resulting software is usually more usable
- User needs are better accommodated
- The design is of higher quality
- The resulting software is easier to maintain
- Overall, the development incurs less cost

Prototyping: disadvantages

- For some projects, it is expensive
- Susceptible to over-engineering:
  - Designers start to incorporate sophistications that they could not incorporate in the prototype.

CONCEPTS COVERED

- Reflection on waterfall-based models
- Incremental
- RAD
- Evolutionary

Major difficulties of Waterfall-Based Models

1. Difficulty in accommodating change requests during development.
   - **40% of the requirements change during development**
2. High cost incurred in developing custom applications.
3. **"Heavy weight processes."**

Major difficulties of Waterfall-Based Life Cycle Models

- In waterfall-based models, requirements are determined and documented at the start:
  - **Assumed to be fixed from that point on.**
  - **Long term planning is made based on this.**

"… the assumption that one can specify a satisfactory system in advance, get bids for its construction, have it built, and install it. …this assumption is fundamentally wrong and many software acquisition problems spring from this…" **Frederick Brooks**

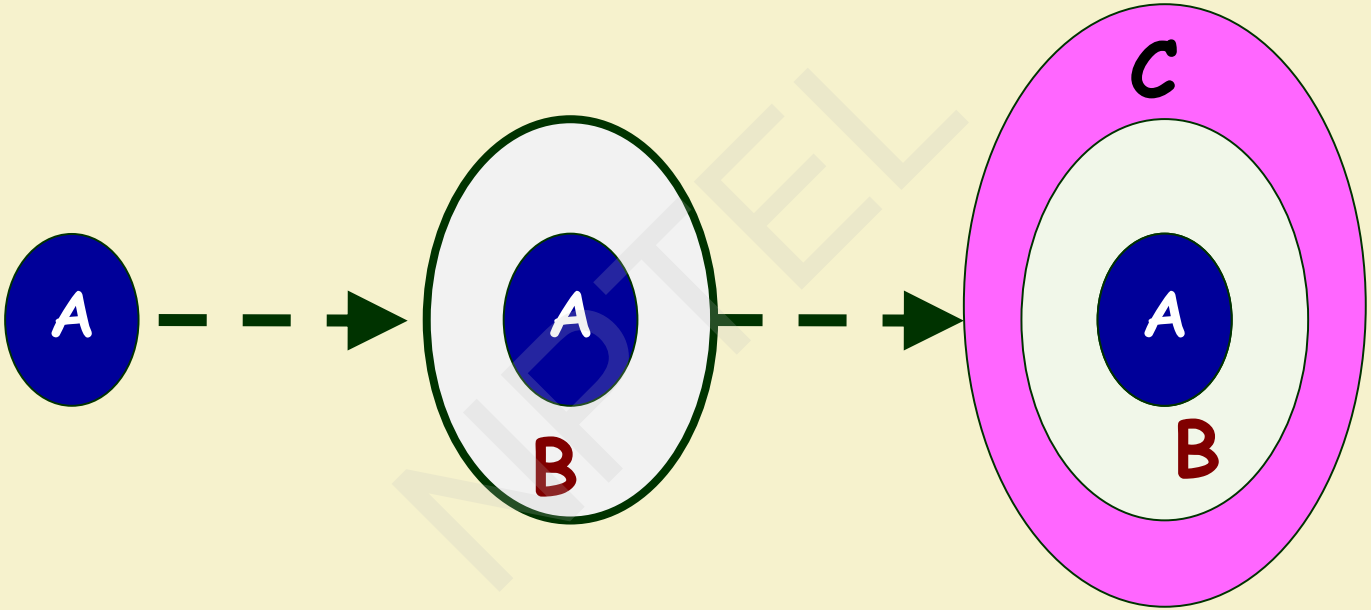
Incremental Model

- “The basic idea… take advantage of what was learned during the development of earlier, incremental, deliverable versions of the system. Learning comes from both the development and use of the system… Start with a simple implementation of a subset of the software requirements and iteratively enhance the evolving sequence of versions. At each version design modifications are made along with adding new functional capabilities. “ **Victor Basili**

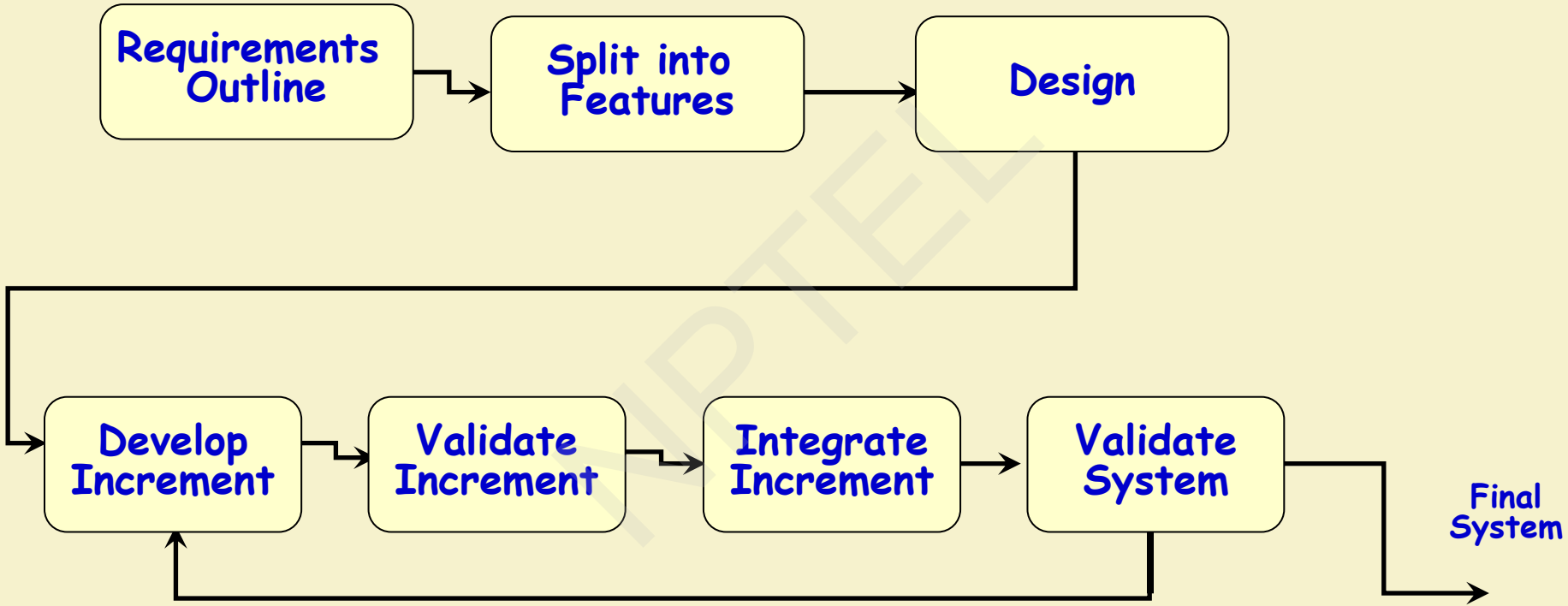
Incremental and Iterative Development (IID)

- **Key characteristics**
  - Builds system incrementally
  - With a number of iterations
  - Each iteration produces a working program
- **Benefits**
  - Facilitates and manages changes
- **Foundation of agile techniques and the basis for**
  - Rational Unified Process (RUP)
  - Extreme Programming (XP)

Customer's Perspective



Incremental Model



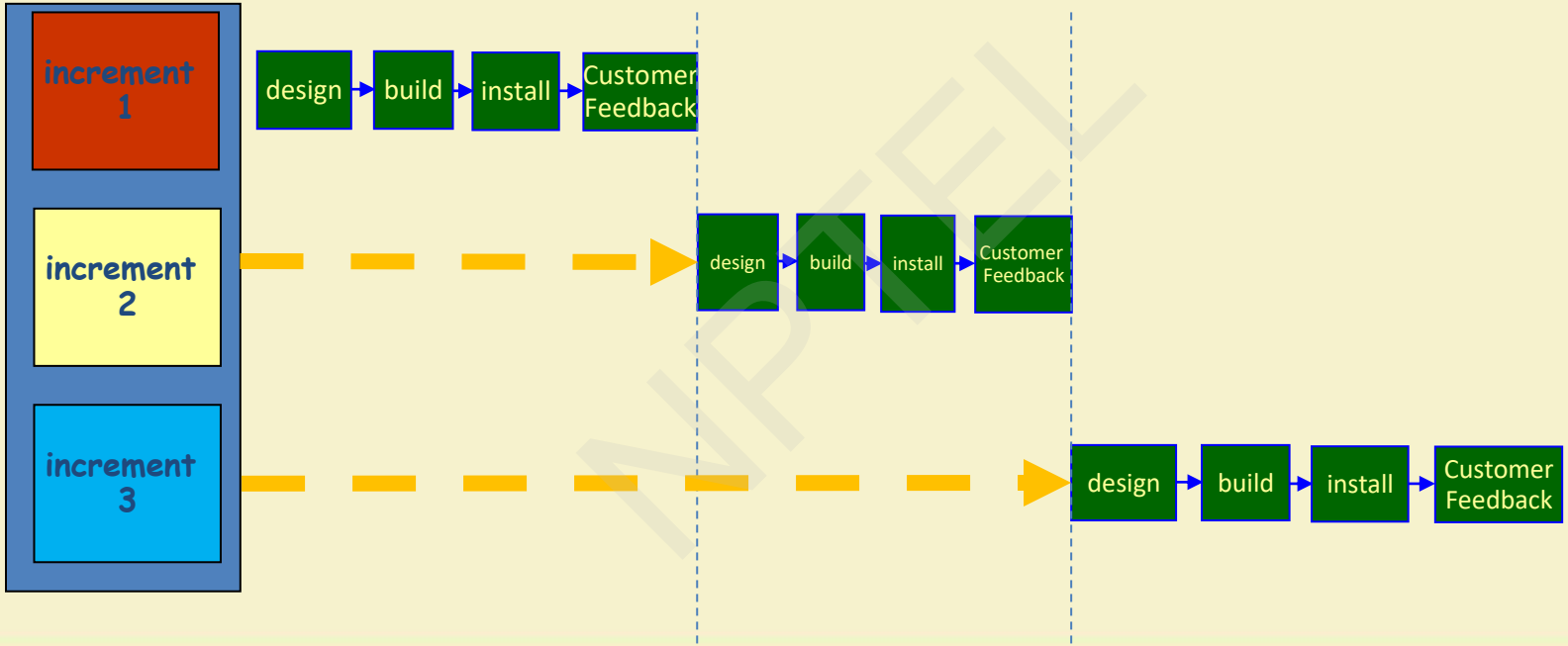
Incremental Model: Requirements

Split into Features

Incremental Model

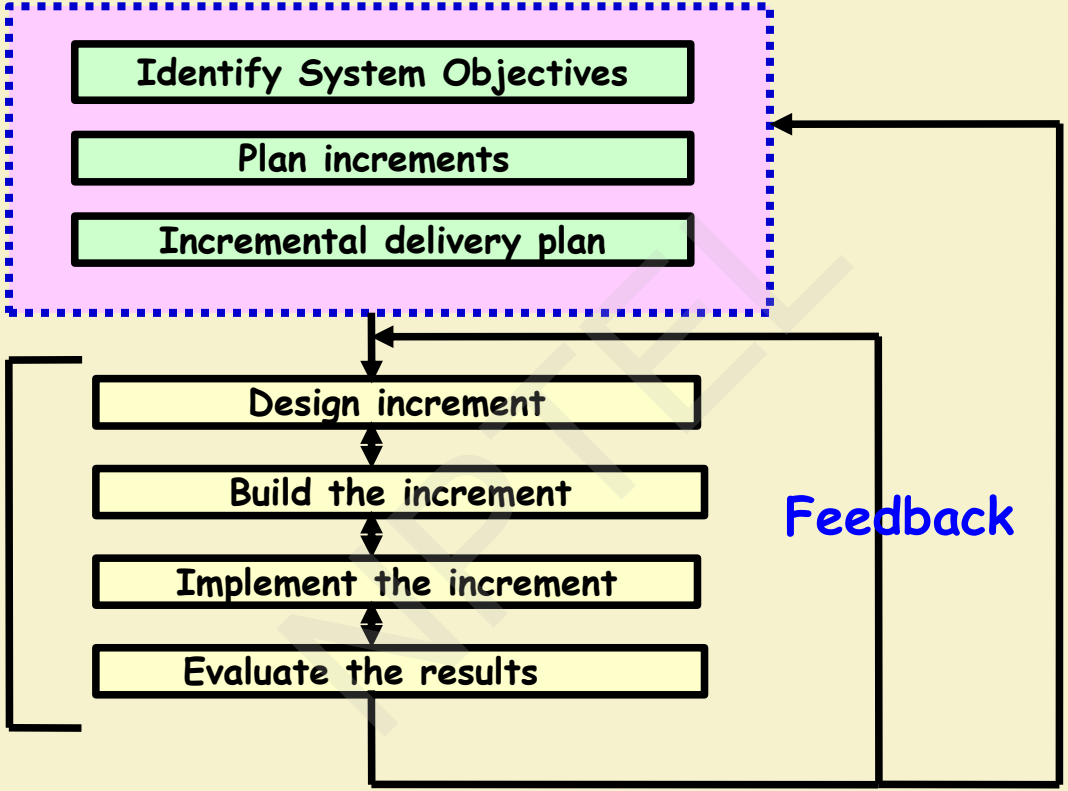
- Waterfall: single release
- Incremental: many releases
  - **First increment: core functionality**
  - **Successive increments: add/fix functionality**
  - **Final increment: the complete product**
- Each iteration: a short mini-project with a separate lifecycle
  - e.g., waterfall

Incremental delivery



The incremental process

Planned incremental delivery



Which step first?

- Some steps will be pre-requisite because of physical dependencies
- Others may be in any order
- Value to cost ratios may be used
  - V/C where
  - V is a score 1-10 representing value to customer
  - C is a score 0-10 representing cost to developers

V/C ratios: an example

step	value	cost	ratio	
profit reports	9	2	4.5	2nd
online database	1	9	0.11	5th
ad hoc enquiry	5	5	1	4th
purchasing plans	9	4	2.25	3rd
profit-based pay for managers	9	1	9	1st

RAD Model

Rapid Application Development (RAD) Model

- Sometimes referred to as the **rapid prototyping model**.
- Major aims:
  - Decrease the time taken and the cost incurred to develop software systems.
  - Facilitate accommodating change requests as early as possible:
    - Before large investments have been made in development and testing.

Important Underlying Principle

- A way to reduce development time and cost, and yet have flexibility to incorporate changes:
  - **Make only short term plans and make heavy reuse of existing code.**

Methodology

- Plans are made for one increment at a time.
  - The time planned for each iteration is called a **time box**.
- Each iteration (increment):
  - Enhances the implemented functionality of the application a little.

Methodology

- During each iteration,
  - A quick-and-dirty prototype-style software for some selected functionality is developed.
  - The customer evaluates the prototype and gives his feedback.
  - The prototype is refined based on the customer feedback.

How Does RAD Facilitate Faster Development?

- RAD achieves fast creation of working prototypes.
  - Through use of specialized tools.
- These specialized tools usually support the following features:
  - **Visual style of development.**
  - **Use of reusable components.**
  - **Use of standard APIs (Application Program Interfaces).**

For which Applications is RAD Suitable?

- Performance and reliability are not critical.

- The system can be split into several independent modules.

For Which Applications RAD is Unsuitable?

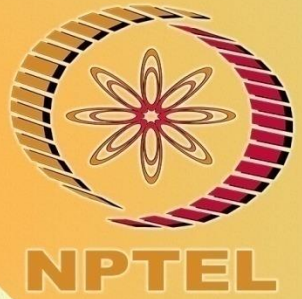
- Few plug-in components are available
- High performance or reliability required
- No precedence for similar products exists
- The system cannot be modularized.

Prototyping versus RAD

- In prototyping model:
  - The developed prototype is primarily used to gain insights into the solution
  - Choose between alternatives
  - Elicit customer feedback.
- The developed prototype:
  - Usually thrown away in prototyping, not in RAD.

CONCEPTS COVERED

- Evolutionary model
- Agile model



Evolutionary Model with Iterations

An Evolutionary and Iterative Development Process...

- Recognizes the reality of changing requirements
  - Capers Jones's research on 8000 projects: **40% of final requirements arrived after development had already begun**
- Promotes early risk mitigation:
  - Breaks down the system into mini-projects and focuses on the riskier issues first.
  - **"plan a little, design a little, and code a little"**
- Encourages all development participants to be involved earlier on,:
  - End users, Testers, integrators, and technical writers

Evolutionary Model with Iteration

- "A complex system will be most successful if implemented in small steps… "retreat" to a previous successful step on failure… opportunity to receive some feedback from the real world before throwing in all resources… and you can correct possible errors…" **Tom Glib** in Software Metrics

Evolutionary model with iteration

- Evolutionary iterative development implies that the requirements, plan, estimates, and solution evolve or are refined over the course of the iterations, rather than fully defined and "frozen" in a major up-front specification effort before the development iterations begin. Evolutionary methods are consistent with the pattern of unpredictable discovery and change in new product development." **Craig Larman**

Evolutionary Model

- First develop the core modules of the software.
- The initial skeletal software is refined into increasing levels of capability: (Iterations)
  - By adding new functionalities in successive versions.

Activities in an Iteration

- Software developed over several “mini waterfalls”.
- The result of a single iteration:
  - –Ends with delivery of some tangible code
  - –An incremental improvement to the software
    --- leads to evolutionary development

Evolutionary Model with Iteration

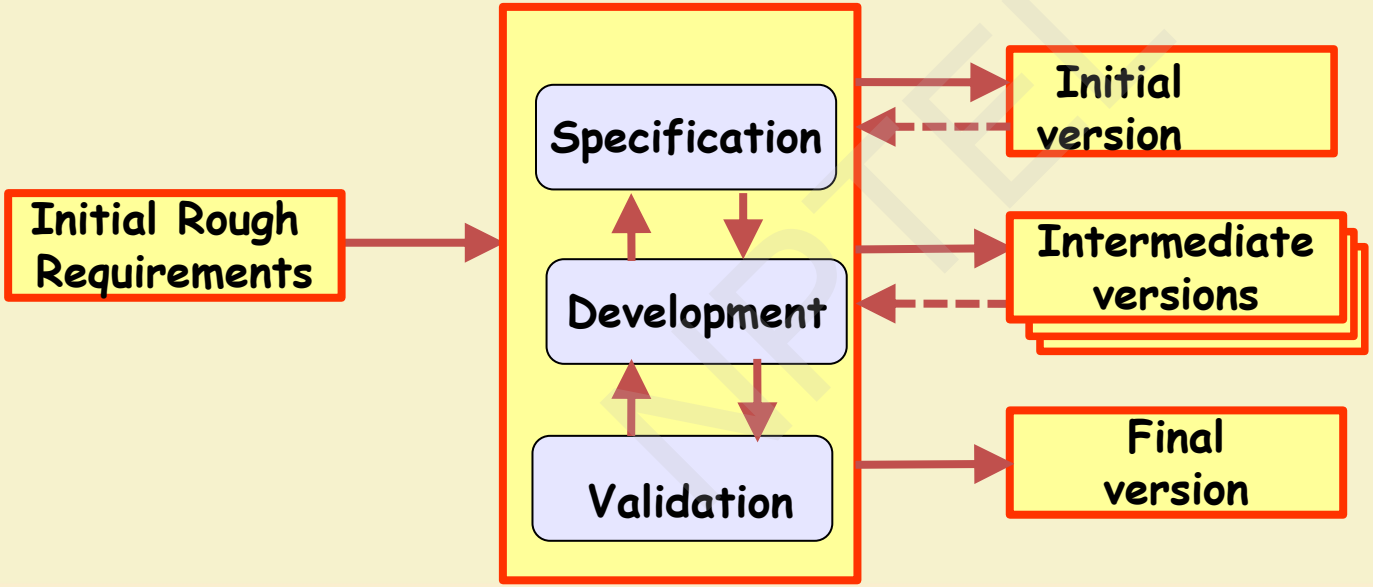
- Outcome of each iteration: tested, integrated, executable system
- Iteration length is short and fixed
  - **Usually between 2 and 6 weeks**
  - **Development takes many iterations (for example: 10-15)**
- Does not "freeze" requirements and then conservatively design :
  - **Opportunity exists to modify requirements as well as the design…**

Evolutionary Model (CONT.)

- Successive versions:
  - Functioning systems capable of performing some useful work.
  - A new release may include new functionality:
    - Also existing functionality in the current release might have been enhanced.

Evolutionary Model

- Evolves an initial implementation with user feedback:
  - **Multiple versions until the final version.**



Advantages of Evolutionary Model

- Users get a chance to experiment with a partially developed system:
  - Much before the full working version is released,
- **Helps finding exact user requirements:**
  - Software more likely to meet exact user requirements.
- **Core modules get tested thoroughly:**
  - Reduces chances of errors in final delivered software.

Advantages of evolutionary model

- Better management of complexity by developing one increment at a time.
- Better management of changing requirements.
- Can get customer feedback and incorporate them much more efficiently:
  - As compared when customer feedbacks come only after the development work is complete.

Advantages of Evolutionary Model with Iteration

- Training can start on an earlier release
- Frequent releases allow developers to fix unanticipated problems quicker.

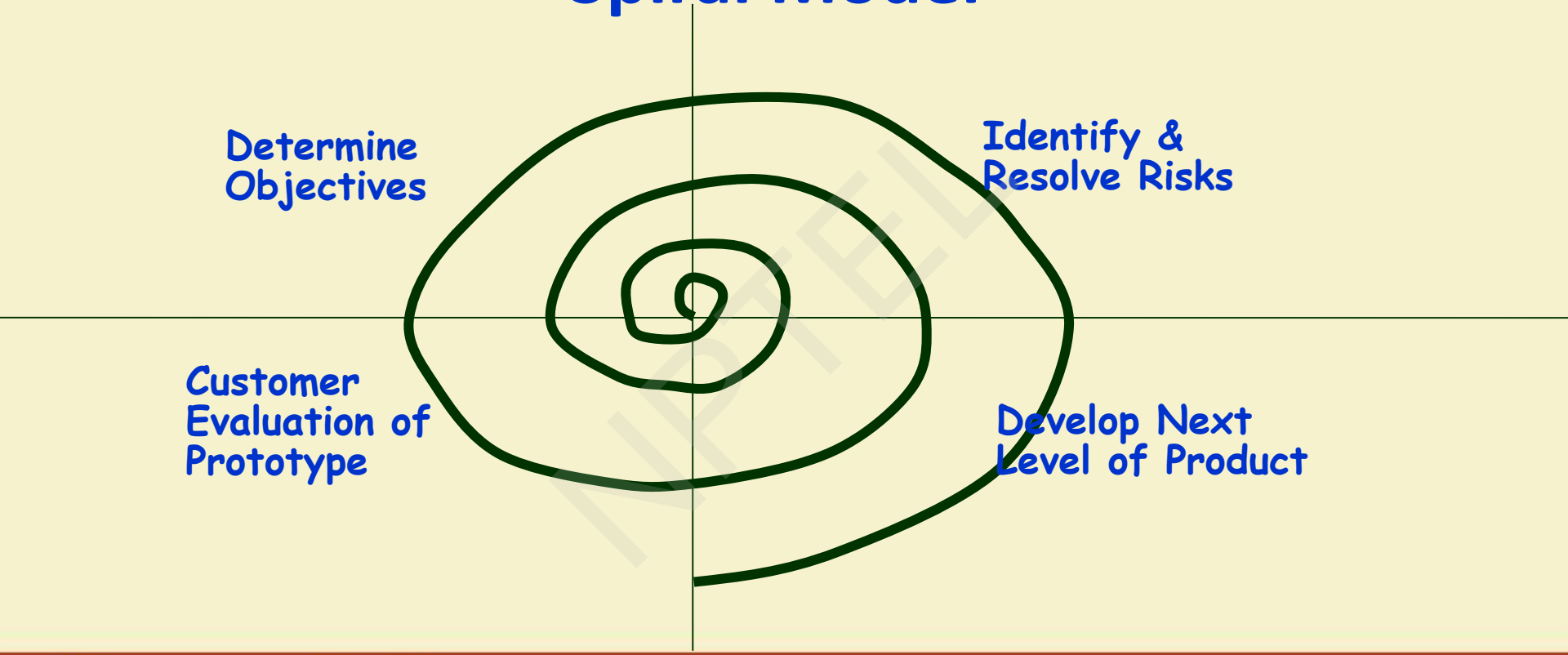
Evolutionary Model: Problems

- **The process is unpredictable:**
  - Hard to manage, e.g., scheduling, workforce allocation, etc.
- **Systems are rather poorly structured:**
  - Continual, unpredictable changes tend to degrade the software structure.
- **Systems may not even converge to a final version.**

Spiral Model

- Proposed by Boehm in 1988.
- Main feature --- risk handling
- Each loop of the spiral represents a phase of the software process:
  - In each phase one or more features taken up, elaborated, risks handled, and implemented
- There are no fixed phases in this model, the phases shown in the figure are just examples.

Spiral Model



Objective Setting (First Quadrant)

- Identify objectives of the phase,
- Examine the risks associated with these objectives.
  - Risk:
    - Any adverse circumstance that might hamper successful completion of a software project.
- Find alternate solutions possible.

Risk Assessment and Reduction (Second Quadrant)

- For each identified project risk,
  - a detailed analysis is carried out.
- Steps are taken to reduce the risk.
- For example, if there is a risk that requirements are inappropriate:
  - A prototype system may be developed.

Spiral Model (CONT.)

- Development and Validation (Third quadrant):
  - develop and validate the next level of the product.
- Review and Planning (Fourth quadrant):
  - review the results achieved so far with the customer and plan the next iteration around the spiral.
- With each iteration around the spiral:
  - progressively more complete version of the software gets built.

- Subsumes all discussed models:
  - a single loop spiral represents waterfall model.
  - uses an evolutionary approach --
    - iterations over the spiral are evolutionary levels.
  - enables understanding and reacting to risks during each iteration along the spiral.
  - Uses:
    - prototyping as a risk reduction mechanism
    - retains the step-wise approach of the waterfall model.

Spiral Model as a Meta Model

Agile Models

What is Agile Software Development?

- Agile: Easily moved, light, nimble, active software processes
- How agility achieved?
  - Fitting the process to the project
  - Avoidance of things that waste time

Agile Model

- Agile model was proposed in mid-1990s
  - To overcome the shortcomings of the waterfall model.
  - Primarily designed to help projects to handle change requests
- **The requirements are decomposed into many small incremental parts that are developed incrementally.**

Ideology: Agile Manifesto

- **Individuals and interactions** *over* process and tools
- **Working Software** *over* comprehensive documentation
- **Customer collaboration** *over* contract negotiation
- **Responding to change** *over* following a plan

`http://www.agilemanifesto.org`

Agile Methodologies

- XP
- Scrum
- Unified process
- Crystal
- DSDM
- Lean

Agile Model: Principal Techniques

- **User stories:**
  - Simpler than use cases.
- **Metaphors:**
  - Based on user stories, developers propose a common vision of what is required.
- **Spike:**
  - Simple program to explore potential solutions.
- **Refactor:**
  - Restructure code without affecting behavior

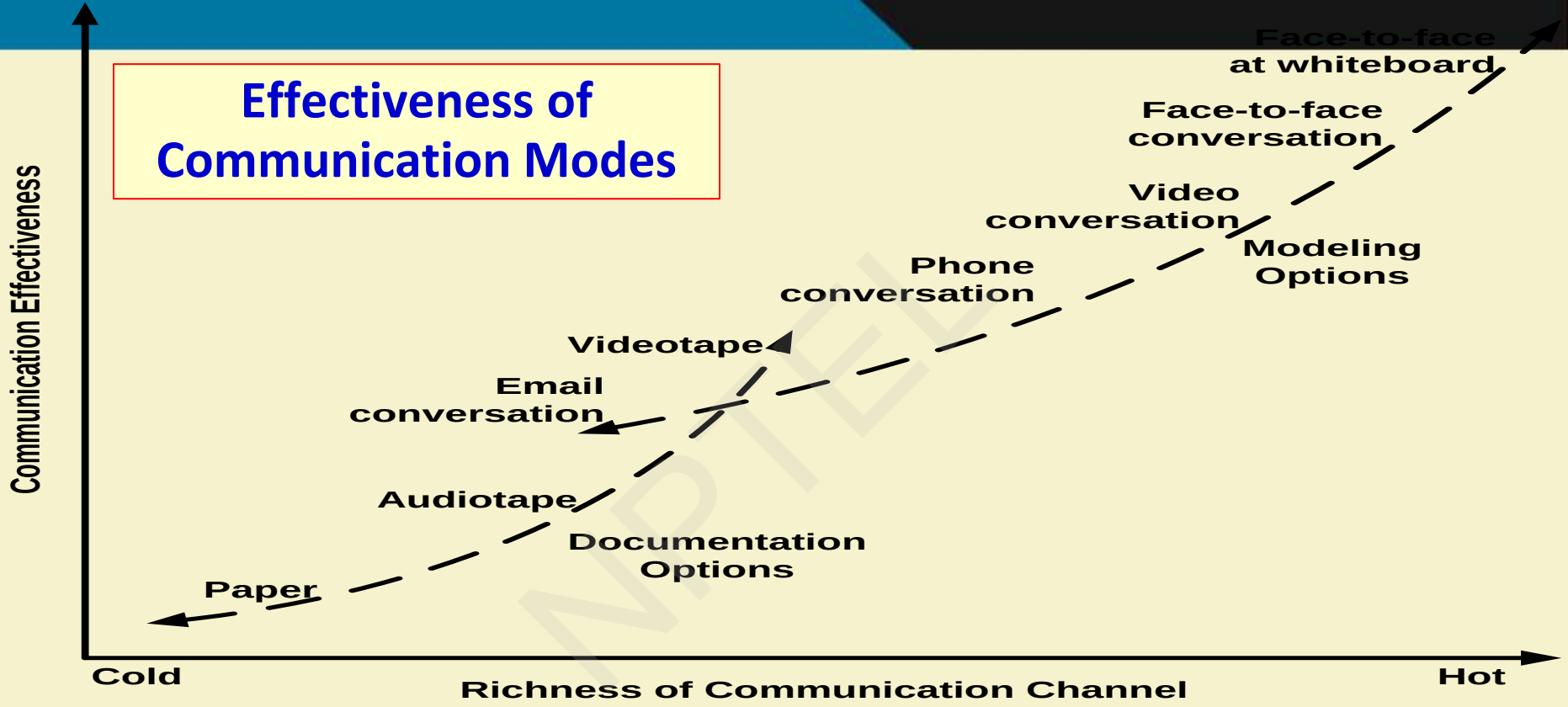
Agile Model: Nitty Gritty

- At a time, only one increment is planned, developed, deployed at the customer site.
  - **No long-term plans are made.**
- An iteration may not add significant functionality,
  - But still a new release is invariably made at the end of each iteration
  - Delivered to the customer for regular use.

Methodology

- **Face-to-face communication favoured over written documents.**
- To facilitate face-to-face communication,
  - Development team to share a single office space.
  - Team size is deliberately kept small (5-9 people)
  - This makes the agile model most suited to small development projects.

Effectiveness of Communication Modes



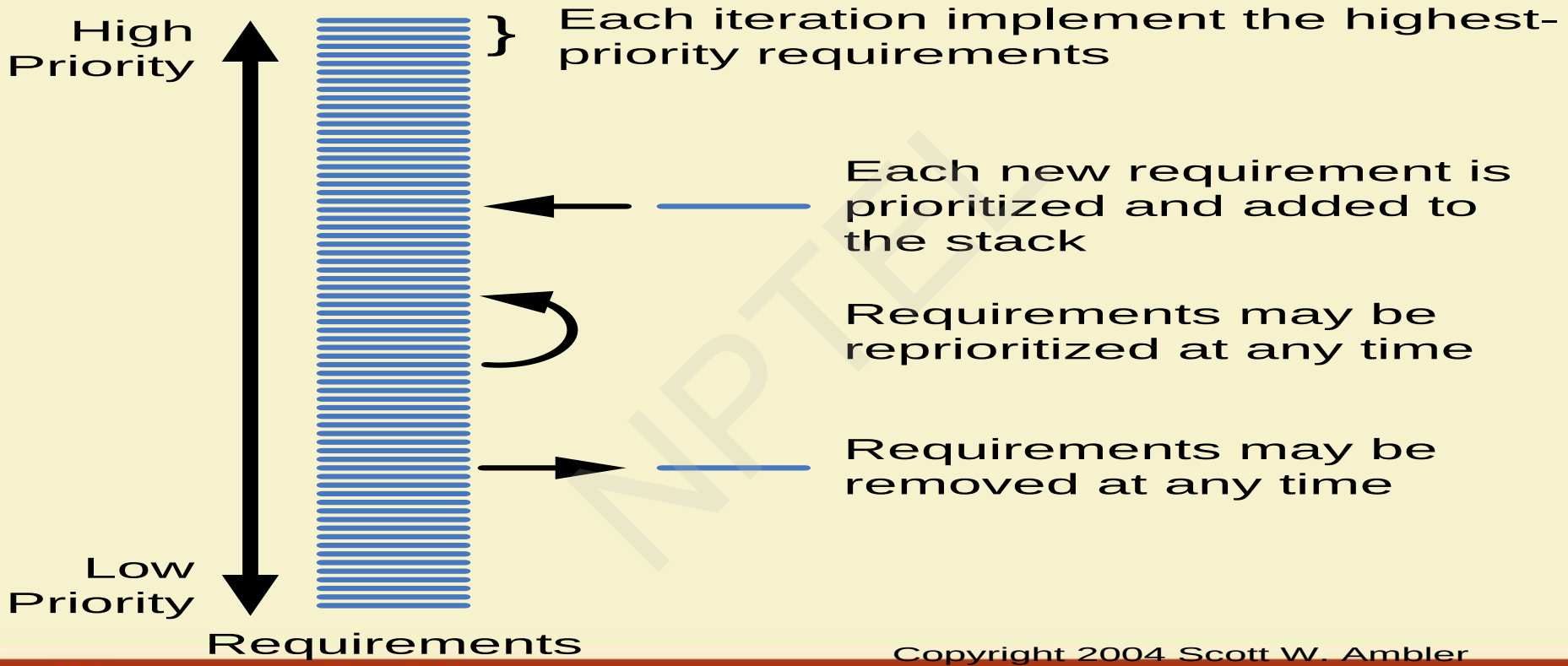
Agile Model: Principles

- The primary measure of progress:
  - **Incremental release of working software**
- Important principles behind agile model:
  - **Frequent delivery of versions --- once every few weeks.**
  - **Requirements change requests are easily accommodated.**
  - **Close cooperation between customers and developers.**
  - **Face-to-face communication among team members.**

Agile Documentation

- **Travel light:**
  - You need far less documentation than you think.
- **Agile documents:**
  - Are concise
  - Describe information that is less likely to change
  - Describe “good things to know”
  - Are sufficiently accurate, consistent, and detailed
- **Valid reasons to document:**
  - Project stakeholders require it
  - To define a contract model
  - To support communication with an external group
  - To think something through

Agile Software Requirements Management



Adoption Detractors

- Sketchy definitions, make it possible to have
  - Inconsistent and diverse definitions
- High quality people skills required
- Short iterations inhibit long-term perspective
- Higher risks due to feature creep:
  - Harder to manage feature creep and customer expectations
  - Difficult to quantify cost, time, quality.

Agile Model Shortcomings

- Derives agility through developing tacit knowledge within the team, rather than any formal document:
  - Can be misinterpreted…
  - External review difficult to get…
  - When project is complete, and team disperses, maintenance becomes difficult…

Extreme Programming (XP)

Extreme Programming Model

- Extreme programming (XP) was proposed by Kent Beck in 1999.
- The methodology got its name from the fact that:
  - **Recommends taking the best practices to extreme levels.**
  - If something is good, why not do it all the time.

Taking Good Practices to Extreme

- **If code review is good:**
  - Always review --- **pair programming**
- **If testing is good:**
  - Continually write and execute test cases --- **test-driven development**
- **If incremental development is good:**
  - Come up with new increments every few days
- **If simplicity is good:**
  - Create the simplest design that will support only the currently required functionality.

Taking to Extreme

- **If design is good,**
  - everybody will design daily (refactoring)
- **If architecture is important,**
  - everybody will work at defining and refining the architecture (metaphor)
- **If integration testing is important,**
  - build and integrate test several times a day (continuous integration)

4 Values

- **Communication**:
  - Enhance communication among team members and with the customers.
- **Simplicity**:
  - Build something simple that will work today rather than something that takes time and yet never used
  - May not pay attention for tomorrow
- **Feedback**:
  - System staying out of users is trouble waiting to happen
- **Courage**:
  - Don’t hesitate to discard code

Best Practices

- **Coding**:
  - without code it is not possible to have a working system.
  - Utmost attention needs to be placed on coding.
- **Testing**:
  - Testing is the primary means for developing a fault-free product.
- **Listening**:
  - Careful listening to the customers is essential to develop a good quality product.

Best Practices

- **Designing**:
  - Without proper design, a system implementation becomes too complex
  - The dependencies within the system become too numerous to comprehend.
- **Feedback**:
  - Feedback is important in learning customer requirements.

Emphasizes Test-Driven Development (TDD)

- Based on user story develop test cases
- Implement a quick and dirty feature every couple of days:
  - **Get customer feedback**
  - **Alter if necessary**
  - **Refactor**
- Take up next feature

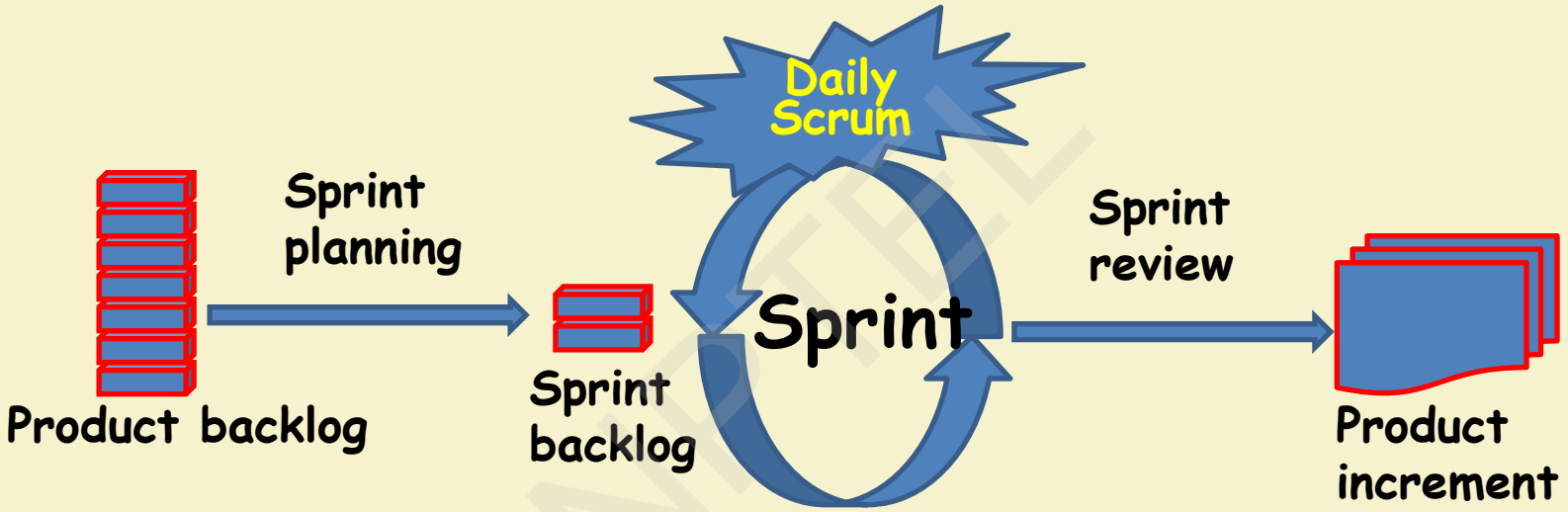
Practice Questions

- What are the stages of iterative waterfall model?
- What are the disadvantages of the iterative waterfall model?
- Why has agile model become so popular?
- What difficulties might be faced if no life cycle model is followed for a certain large project?

Scrum

Scrum: Characteristics

- Self-organizing teams
- Product progresses in a series of month-long **sprints**
- Requirements are captured as items in a list of **product backlog**
- One of the agile processes


Daily
Scrum
Sprint
planning
Sprint
review
Sprint
Product backlog
Sprint
backlog
Product
increment

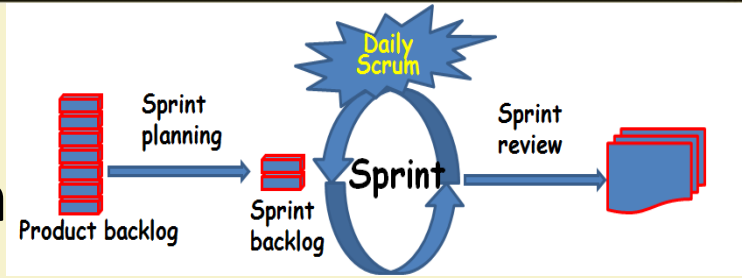
Sprint

- Scrum projects progress in a series of “sprints”
  - Analogous to XP iterations or time boxes
  - Target duration is one month
- **Software increment is designed, coded, and tested during the sprint**
- **No changes entertained during a sprint**

CONCEPTS COVERED

- Scrum
- Selecting an SDLC
- Project Scheduling

Sprint



- Fundamental process flow of Scrum
- A month-long iteration, during which an incremental product functionality completed
- NO outside influence allowed to interfere with the Scrum team during a Sprint

Scrum Framework

- **Roles:** Product Owner, ScrumMaster, Team
- **Ceremonies:** Sprint Planning, Sprint Review, Sprint Retrospective, and Daily Scrum Meeting
- **Artifacts:** Product Backlog, Sprint Backlog, and Burndown Chart

Key Roles and Responsibilities in Scrum Process

- **Product Owner**
  - Acts on behalf of customers to represent their interests.
- **Development Team**
  - Team of five-nine people with cross-functional skill sets.
- **Scrum Master (aka Project Manager)**
  - Facilitates scrum process and resolves impediments at the team and organization levels
  - Acts as a buffer between the team and outside interference.

Product Owner

- Defines the features of the product
- Decides on release date and features
- Prioritizes features according to market value
- Adjusts features and priority every iteration, as needed
- Accepts or rejects the work results.

The Scrum Master

- Essentially the project manager
- Removes impediments
- Ensures that the team is fully functional and productive
- Enables close cooperation across all roles and functions
- Shields the team from external interferences

Scrum Team

- Typically 5-10 people
- Cross-functional
  - QA, Programmers, testers, UI Designers, etc.
- Teams are self-organizing
- Membership can change only between sprints

Ceremonies

- Sprint Planning Meeting
- Daily Scrum
- Sprint retrospective
- Sprint Review Meeting

Sprint Planning

- Goal is to produce Sprint Backlog
- Product owner along with the Team negotiate the Backlog Items that the team will work on in order to meet Release Goals
- Scrum Master ensures Team agrees to realistic goals

- Daily
- 15-minutes
- Stand-up meeting
- Not for problem solving
- Three questions:
  1. What did you do yesterday?
  2. What will you do today?
  3. What obstacles are in your way?

Daily Scrum

Daily Scrum

- Is NOT a problem solving session
- Is NOT a way to collect information about WHO is behind the schedule
- Is a meeting in which team members make commitments to each other and to the Scrum Master
- Is a good way for a Scrum Master to track the progress of the Team

- Team presents what it accomplished during the sprint
- Typically takes the form of a demo of new features
- Informal
  - 2-hour prep time rule
- Participants
  - Customers
  - Management
  - Product Owner
  - Team members

Sprint Review Meeting

Product Backlog

Product backlog

- A list of all desired work on the project
  - Usually a combination of
    - **story-based work (“let user search and replace”)**
    - **task-based work (“improve exception handling”)**
- List is prioritized by the Product Owner

Product Backlog

- Requirements for a system, expressed as a prioritized list of Backlog Items
  - **Managed and owned by Product Owner**
  - **Typically Spreadsheet**
  - **Usually is created during the Sprint Planning Meeting**

	Item #	Description	Est	By
Very High				
	1	**Finish database versioning**	16	KH
	2	**Get rid of unneeded shared Java in database**	8	KH
	-	**Add licensing**	-	-
	3	Concurrent user licensing	16	TG
	4	Demo / Eval licensing	16	TG
		Analysis Manager		
	5	File formats we support are out of date	160	TG
	6	Round-trip Analyses	250	MC
High				
	-	**Enforce unique names**	-	-
	7	In main application	24	KH
	8	In import	24	AM
	-	**Admin Program**	-	-
	9	Delete users	4	JM
	-	**Analysis Manager**	-	-
	10	When items are removed from an analysis, they should show up again in the pick list in lower 1/2 of the analysis tab	8	TG
	-	**Query**	-	-
	11	Support for wildcards when searching	16	T&A
	12	Sorting of number attributes to handle negative numbers	16	T&A
	13	Horizontal scrolling	12	T&A
	-	**Population Genetics**	-	-
	14	Frequency Manager	400	T&M
	15	Query Tool	400	T&M
	16	Additional Editors (which ones)	240	T&M
	17	Study Variable Manager	240	T&M
	18	Haplotypes	320	T&M
	19	**Add icons for v1.1 or 2.0**	-	-
	-	**Pedigree Manager**	-	-
	20	Validate Derived kindred	4	KH
Medium				
	-	**Explorer**	-	-
	21	Launch tab synchronization (only show queries/analyses for logged in users)	8	T&A
	22	Delete settings (?)	4	T&A

Sample Product Backlog

Sprint Backlog

- A subset of Product Backlog Items, which define the work for a Sprint
  - **Created by Team members**
  - **Updated daily**

Sprint Backlog during the Sprint

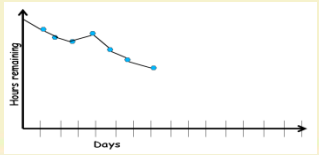
- Changes occur:
  - Team adds new tasks whenever they need to in order to meet the Sprint Goal
  - Team can remove unnecessary tasks
  - But: Sprint Backlog can only be updated by the team
- Estimates are updated whenever there's new information

Burn down Charts

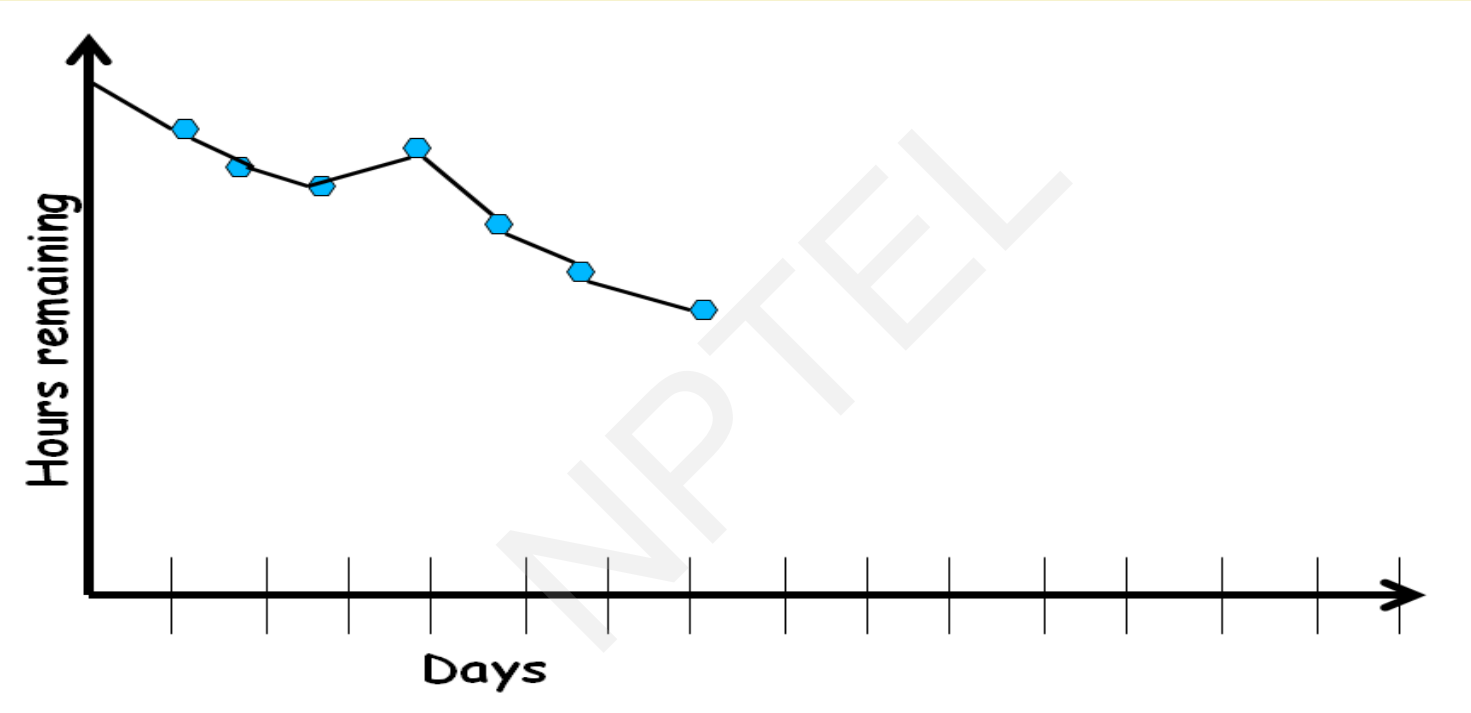
- Are used to represent “work done”.
- Are remarkably simple but effective Information disseminators
- 3 Types:
  - **Sprint Burn down Chart (progress of the Sprint)**
  - **Release Burn down Chart (progress of release)**
  - **Product Burn down chart (progress of the Product)**

Sprint Burn down Chart

- Depicts the total Sprint Backlog hours remaining per day
- Shows the estimated amount of time to release
- Ideally should burn down to zero to the end of the Sprint
- In reality is not a straight line

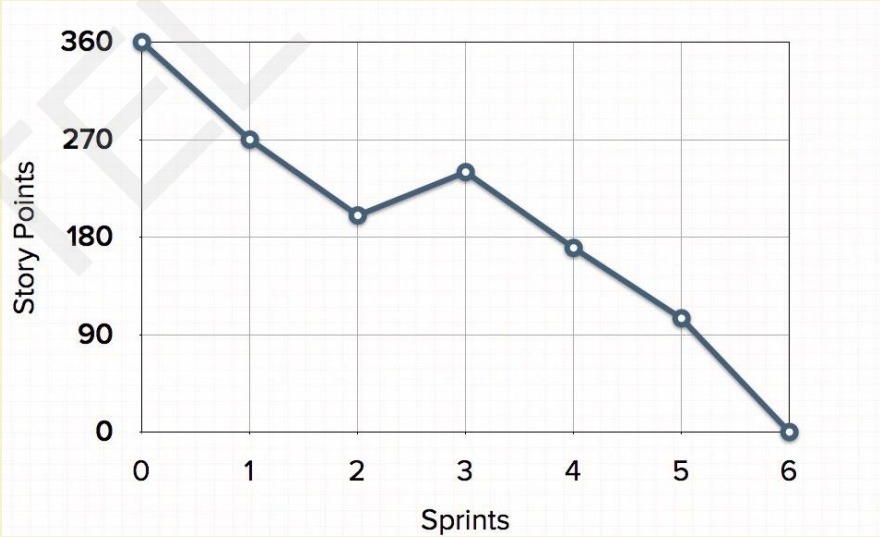


Sprint Burndown Chart



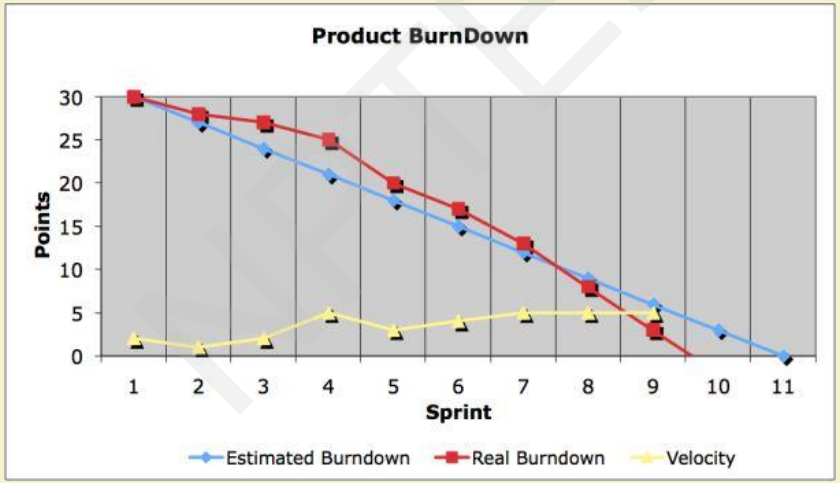
Release Burndown Chart

- Will the final release be done on right time?
- How many more sprints?
- X-axis: sprints
- Y-axis: amount of story points remaining



Product Burndown Chart

- Is a “big picture” view of project’s progress (all the releases)



Scalability of Scrum

- A typical Scrum team is 6-10 people
- Jeff Sutherland - up to over 800 people
- "Scrum of Scrums" or what called "Meta-Scrum“
- Frequency of meetings is based on the degree of coupling between packets

Selecting A Process

- Appropriate process model depends on:
  - **Characteristics of product, development team, and customer**
- Whenever uncertainty is high, evolutionary approach is favored.
- For well understood applications, waterfall model is desirable.
- For novice team members, incremental model is preferable,


Thank you